

Name: Turpati Dinesh Kumar
Regd. No: 2141011200

System Monitor Tool

Objective: Create a system monitor tool in C++ that displays real-time information about system processes, memory usage, and CPU load, similar to the 'top' command.

Day-wise Tasks:

Day 1: Design UI layout and gather system data using system calls.

Code:

```
#include <iostream>

using namespace std;

#include <sys/sysinfo.h>

void displayMemoryInfo(){

    struct sysinfo info;

    if (sysinfo(&info)==0){

        cout<< "Total used RAM (info.totalram in MB):
"<<info.totalram/(1024*1024)<<"MB\n";

        cout<< "Total used RAM (info.totalram in GB):
"<<(info.totalram/(1024*1024))/1024<<"GB\n";

        cout<< "Total unused RAM (info.freeram in MB):
"<<info.freeram/(1024*1024)<<"MB\n";

        cout<< "Total shared RAM (info.sharedram in MB):
"<<info.sharedram/(1024*1024)<<"MB\n";

    }

}

int main(){

    displayMemoryInfo();

    return 0;

}
```

Output:

```
student@D001-38: ~/Desktop/2141011200_LSP
student@D001-38:~/Desktop/2141011200_LSP$ gedit task1.cpp
student@D001-38:~/Desktop/2141011200_LSP$ g++ --std=c++17 task1.cpp -o task1
student@D001-38:~/Desktop/2141011200_LSP$ ./task1
Total used RAM (info.totalram in MB): 7681MB
Total used RAM (info.totalram in GB): 7GB
Total unused RAM (info.freeram in MB): 509MB
Total shared RAM (info.sharedram in MB): 209MB
student@D001-38:~/Desktop/2141011200_LSP$
```

Day 2: Display process list with CPU and memory usage.

Code:

```
#include <iostream>

#include <fstream>

#include <sstream>

#include <thread>

#include <chrono>

using namespace std;

struct CPUData{

    long user, nice, system, idle, iowait, irq, softirq, steal, guest, guest_nice;

};

CPUData getCPUData(){

    ifstream file("/proc/stat");

    string line;

    CPUData cpu={};

    if (file.is_open()){

        getline(file,line);

        istringstream ss(line);

        string cpuLabel;

        ss >> cpuLabel >> cpu.user >> cpu.nice >> cpu.system >> cpu.idle >> cpu.iowait >>

cpu.irq >> cpu.softirq >> cpu.steal >> cpu.guest >> cpu.guest_nice;

    }

    return cpu;

}
```

```

}

double calculateCPUUsage (CPUData prev, CPUData current){
    long prevIdle= prev.idle + prev.iowait;
    long currIdle= current.idle + current.iowait;

    long prevTotal = prev.user + prev.nice + prev.system + prev.idle + prev.iowait + prev.irq +
prev.softirq + prev.steal;

    long currTotal = current.user + current.nice + current.system + current.idle + current.iowait +
current.irq + current.softirq + current.steal;

    long totalDiff = currTotal - prevTotal;
    long idleDiff = currIdle - prevIdle;

    return ((totalDiff - idleDiff) * 100.0) / totalDiff;
}

int main(){
    CPUData cpu= getCPUData();

    cout << "User: " << cpu.user << endl;
    cout << "Nice: " << cpu.nice << endl;
    cout << "System: " << cpu.system << endl;
    cout << "Idle: " << cpu.idle << endl;
    cout << "IOwait: " << cpu.iowait << endl;
    cout << "IRQ: " << cpu.irq << endl;
    cout << "SoftIRQ: " << cpu.softirq << endl;
    cout << "Steal: " << cpu.steal << endl;
    cout << "Guest: " << cpu.guest << endl;
    cout << "Guest Nice: " << cpu.guest_nice << endl;

    CPUData prevData = getCPUData();
    this_thread::sleep_for(chrono::seconds(1));
    CPUData currData = getCPUData();

    double cpuUsage= calculateCPUUsage (prevData, currData);

    cout << "CPU Usage: "<<cpuUsage<<endl;

    return 0;
}

```

Output:

```

student@D001-38: ~/Desktop/2141011200_LSP/Task-2
student@D001-38:~/Desktop/2141011200_LSP/Task-2$ gedit task2.cpp
student@D001-38:~/Desktop/2141011200_LSP/Task-2$ g++ --std=c++17 task2.cpp -o task2
student@D001-38:~/Desktop/2141011200_LSP/Task-2$ ./task2
User: 45292
Nice: 405
System: 12704
Idle: 2614125
IOWait: 13434
IRQ: 0
SoftIRQ: 1929
Steal: 0
Guest: 0
Guest Nice: 0
CPU Usage: 0.417014
student@D001-38:~/Desktop/2141011200_LSP/Task-2$ gedit task2.cpp

```

```

student@D001-38:~/Desktop/2141011200_LSP/Task-2$ gedit task2.cpp
student@D001-38:~/Desktop/2141011200_LSP/Task-2$ g++ --std=c++17 task2.cpp -o task2
student@D001-38:~/Desktop/2141011200_LSP/Task-2$ ./task2_
bash: ./task2_: No such file or directory
student@D001-38:~/Desktop/2141011200_LSP/Task-2$ ./task2
Active processes:
PID: 1|Name: (systemd)
PID: 2|Name: (kthread)
PID: 3|Name: (rcu_gp)
PID: 4|Name: (rcu_par_gp)
PID: 5|Name: (slub_flushwq)
PID: 6|Name: (netns)
PID: 8|Name: (kworker/0:0H-events_highpri)
PID: 10|Name: (mm_percpu_wq)
PID: 11|Name: (rcu_tasks_rude_)
PID: 12|Name: (rcu_tasks_trace)
PID: 13|Name: (ksoftirqd/0)
PID: 14|Name: (rcu_sched)
PID: 15|Name: (migration/0)
PID: 16|Name: (idle_inject/0)
PID: 17|Name: (kworker/0:1-events)
PID: 18|Name: (cpuhp/0)
PID: 19|Name: (cpuhp/1)
PID: 20|Name: (idle_inject/1)
PID: 21|Name: (migration/1)
PID: 22|Name: (ksoftirqd/1)
PID: 24|Name: (kworker/1:0H-kblockd)
PID: 25|Name: (cpuhp/2)
PID: 26|Name: (idle_inject/2)
PID: 27|Name: (migration/2)
PID: 28|Name: (ksoftirqd/2)
PID: 30|Name: (kworker/2:0H-events_highpri)
PID: 31|Name: (cpuhp/3)
PID: 32|Name: (idle_inject/3)
PID: 33|Name: (migration/3)
PID: 34|Name: (ksoftirqd/3)
PID: 36|Name: (kworker/3:0H-events_highpri)
PID: 37|Name: (cpuhp/4)
PID: 38|Name: (idle_inject/4)
PID: 39|Name: (migration/4)
PID: 40|Name: (ksoftirqd/4)
PID: 42|Name: (kworker/4:0H-kblockd)
PID: 43|Name: (cpuhp/5)
PID: 44|Name: (idle_inject/5)
PID: 45|Name: (migration/5)
PID: 46|Name: (ksoftirqd/5)

```

Day 3: Implement process sorting by CPU and memory usage.

Code:

```

#include <iostream>

#include <filesystem>

#include <fstream>

#include <sstream>

#include <algorithm>

```

```

#include <vector>

#include <dirent.h>

#include <unistd.h>

#include <csignal>

using namespace std;

namespace fs=std::filesystem;

struct ProcessInfo{

    int pid;

    string name;

    double cpuUsage;

    long memoryUsage;

};

string readFileValue(const string &path){

    ifstream file(path);

    string value;

    if (file.is_open()){

        getline(file,value);

    }

    return value;

}

double getSystemUptime(){

    ifstream file("/proc/uptime");

    double uptime;

    if (file.is_open()){

        file >> uptime;

    }

    return uptime;

}

ProcessInfo getProcessInfo(int pid,double systemUptime){

    ProcessInfo pinfo;

    pinfo.pid = pid;

    ifstream file("/proc/" + to_string(pid) + "/stat");

    string line;

```

```

long utime=0, stime=0, starttime=0;

if(file.is_open()){
    getline(file,line);
    istringstream ss(line);
    string token;
    int count=0;

    while(ss >> token){
        count++;
        if(count==2) pinfo.name=token;
        else if(count==14) utime=stol(token);
        else if(count==15) stime=stol(token);
        else if(count==22) starttime=stol(token);
    }
}

ifstream memFile("/proc/" + to_string(pid) + "/status");
if (memFile.is_open()){
    string key, value,unit;
    while(memFile >> key >> value >> unit){
        if (key=="VmRSS"){
            pinfo.memoryUsage=stol(value);
            break;
        }
    }
}

long total_time = utime + stime;

double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));
pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;
return pinfo;

}

```

```

vector <ProcessInfo> getAllProcesses(){
    vector <ProcessInfo> processes;
    double systemUptime= getSystemUptime();
    for (const auto &entry : fs::directory_iterator("/proc")){
        if (entry.is_directory()){
            string filename=entry.path().filename().string();
            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);
                processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }
    return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)
        {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

```

```

int main(){

    vector <ProcessInfo> processes = getAllProcesses();

    sortProcesses(processes,true);

    cout<<"PID\tCPU%\tMemory (kB)\tName\n";

    for (size_t i=0;i<min(processes.size(),size_t(10));i++){

        cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUsage<
        <"%\t"<<processes[i].name<<"%\n";

        }

    return 0;

}

```

Output:

```

student@0001-38:~/Desktop/2141011200_LSP/Task3$ g++ --std=c++17 task3.cpp -o task3
student@0001-38:~/Desktop/2141011200_LSP/Task3$ ./task3
PID      CPU%    Memory (kB)   Name
1452     4.32428%    140730319663064% (gnome-shell)%
826      4.02439%    140730319663064% (snapd)%
1117     3.48917%    140730319663064% (Xorg)%
6727     2.99989%    140730319663064% (chrome)%
6906     1.94707%    140730319663064% (nautilus)%
6524     0.990062%   140730319663064% (chrome)%
1099     0.8633%    140730319663064% (mysqld)%
6709     0.623418%   140730319663064% (chrome)%
6567     0.495031%   140730319663064% (chrome)%
558      0.466781%   140730319663064% (irq/152-iwlwifi)%
student@0001-38:~/Desktop/2141011200_LSP/Task3$

```

Day 4: Add functionality to kill processes.

Code:

```

#include <iostream>

#include <filesystem>

#include <fstream>

#include <sstream>

#include <algorithm>

#include <vector>

#include <dirent.h>

#include <unistd.h>

#include <csignal>

```



```

using namespace std;

namespace fs=std::filesystem;

struct ProcessInfo{
    int pid;
    string name;
    double cpuUsage;
    long memoryUsage;
};

string readFileValue(const string &path){
    ifstream file(path);
    string value;
    if (file.is_open()){
        getline(file,value);
    }
    return value;
}

double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
        file >> uptime;
    }
    return uptime;
}

ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;
    pinfo.pid = pid;
    ifstream file("/proc/" + to_string(pid) + "/stat");
    string line;
    long utime=0, stime=0, starttime=0;
    if(file.is_open()){
        getline(file,line);
        istringstream ss(line);
    }
}

```

```

        string token;

        int count=0;

        while(ss >> token){

            count++;

            if(count==2) pinfo.name=token;

            else if(count==14) utime=stol(token);

            else if(count==15) stime=stol(token);

            else if(count==22) starttime=stol(token);

        }

    }

    ifstream memFile("/proc/" + to_string(pid) + "/status");

    if (memFile.is_open()){

        string key, value,unit;

        while(memFile >> key >> value >> unit){

            if (key=="VmRSS"){

                pinfo.memoryUsage=stol(value);

                break;

            }

        }

    }

    long total_time = utime + stime;

    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));

    pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;

    return pinfo;

}

vector <ProcessInfo> getAllProcesses(){

    vector <ProcessInfo> processes;

    double systemUptime= getSystemUptime();

```

```

for (const auto &entry : fs::directory_iterator("/proc")){
    if (entry.is_directory()){
        string filename=entry.path().filename().string();
        if (all_of(filename.begin(),filename.end(), ::isdigit)){
            int pid=stoi(filename);
            processes.push_back(getProcessInfo(pid,systemUptime));
        }
    }
}
return processes;
}

```

```

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)
        {
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

```

```

int main(){
    vector <ProcessInfo> processes = getAllProcesses();
    sortProcesses(processes,true);
    cout<<"PID\tCPU%\tMemory (kB)\tName\n";
}

```

```

        for (size_t i=0;i<min(processes.size(),size_t(10));i++){

            cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUsage<
            <"%\t"<<processes[i].name<<"%\n";

        }

        int targetPid;

        cout<<"Enter PID to kill: ";

        cin>>targetPid;

        if(targetPid > 0){

            if (kill(targetPid, SIGKILL) == 0){

                cout<<"Process " << targetPid << " terminated successfully\n";

            }

            else{

                perror("Failed to kill the process");

            }

        }

        return 0;

    }

```

Output:

```

student@0001-38:~/Desktop/2141011200_LSP/Task-4$ gedit task4.cpp
student@0001-38:~/Desktop/2141011200_LSP/Task-4$ g++ -std=c++17 task4.cpp -o task4
student@0001-38:~/Desktop/2141011200_LSP/Task-4$ ./task4
PID      CPU%    Memory (kB)    Name
1452     4.25945%    140725246120248%    (gnome-shell)%
826      3.68838%    140725246120248%    (snapd)%
1117     3.46128%    140725246120248%    (Xorg)%
6727     2.65766%    140725246120248%    (chrome)%
6906     1.85079%    140725246120248%    (nautilus)%
1099     0.890043%   140725246120248%    (mysqld)%
6524     0.847809%   140725246120248%    (chrome)%
6709     0.663155%   140725246120248%    (chrome)%
6567     0.565206%   140725246120248%    (chrome)%
6496     0.44541%    140725246120248%    (gnome-terminal-)%
Enter PID to kill: 6906
Process 6906 terminated successfully
student@0001-38:~/Desktop/2141011200_LSP/Task-4$ |

```

Day 5: Implement real-time update feature to refresh data every few seconds.

Code:

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <vector>
#include <dirent.h>
#include <unistd.h>
#include <csignal>
#include <thread>
#include <chrono>

using namespace std;
namespace fs=std::filesystem;

struct ProcessInfo{
    int pid;
    string name;
    double cpuUsage;
    long memoryUsage;
};

string readFileValue(const string &path){
    ifstream file(path);
    string value;
    if (file.is_open()){
        getline(file,value);
    }
    return value;
}

double getSystemUptime(){
    ifstream file("/proc/uptime");
    double uptime;
    if (file.is_open()){
```

```

        file >> uptime;

    }

    return uptime;
}

ProcessInfo getProcessInfo(int pid,double systemUptime){
    ProcessInfo pinfo;

    pinfo.pid = pid;

    ifstream file("/proc/" + to_string(pid) + "/stat");

    string line;

    long utime=0, stime=0, starttime=0;

    if(file.is_open()){
        getline(file,line);

        istringstream ss(line);

        string token;

        int count=0;

        while(ss >> token){
            count++;

            if(count==2) pinfo.name=token;

            else if(count==14) utime=stol(token);

            else if(count==15) stime=stol(token);

            else if(count==22) starttime=stol(token);

        }

    }

    ifstream memFile("/proc/" + to_string(pid) + "/status");

    if (memFile.is_open()){
        string key, value,unit;

        while(memFile >> key >> value >> unit){
            if (key=="VmRSS"){
                pinfo.memoryUsage=stol(value);

                break;
            }
        }
    }
}

```

```

        }

    }

    long total_time = utime + stime;

    double seconds = systemUptime - (starttime / sysconf(_SC_CLK_TCK));

    pinfo.cpuUsage = (total_time / sysconf(_SC_CLK_TCK)) / seconds * 100;

    return pinfo;

}

vector<ProcessInfo> getAllProcesses(){
    vector<ProcessInfo> processes;

    double systemUptime= getSystemUptime();

    for (const auto &entry : fs::directory_iterator("/proc")){
        if (entry.is_directory()){
            string filename=entry.path().filename().string();

            if (all_of(filename.begin(),filename.end(), ::isdigit)){
                int pid=stoi(filename);

                processes.push_back(getProcessInfo(pid,systemUptime));
            }
        }
    }

    return processes;
}

void sortProcesses(vector<ProcessInfo> &processes, bool sortByCpu){
    if(sortByCpu){
        sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
&b)
        {
            return a.cpuUsage > b.cpuUsage;
        });
    }
    else{

```

```

&b)          sort(processes.begin(),processes.end(),[](const ProcessInfo &a, const ProcessInfo
{
            return a.memoryUsage > b.memoryUsage;
        });
    }
}

```

```

int main(){
    char input;
    while (true){
        system("clear");
        vector <ProcessInfo> processes = getAllProcesses();
        sortProcesses(processes,true);
        cout<<"Press 'q' to quit";
        cout<<"PID\tCPU%\tMemory (kB)\tName\n";
        for (size_t i=0;i<min(processes.size(),size_t(10));i++){

            cout<<processes[i].pid<<"\t"<<processes[i].cpuUsage<<"%\t"<<processes[i].memoryUsage<
            <<"%\t"<<processes[i].name<<"%\n";

        }

        int targetPid;
        cout<<"Enter PID to kill: ";
        cin>>targetPid;
        if (targetPid == 0){
            //continue refresh
        }
        else if(targetPid > 0){
            if (kill(targetPid, SIGKILL) == 0){
                cout<<"Process "<< targetPid << " terminated successfully\n";
            }
            else{
                perror("Failed to kill the process");
            }
        }
    }
}

```



```

    }

}

cout << "\nPress 'q' to quit or Enter to refresh: ";

cin.ignore( );

input = getchar( );

if (input == 'q' || input == 'Q')

break;

this_thread::sleep_for(chrono::seconds(2));

}

return 0;

}

```

Output:

```

student@D001-38: ~/Desktop/Z141011200_LSP/Task-5

Press 'q' to quitPID    CPU%    Memory (kB)    Name
1452  4.0723% 140722276908360% (gnome-shell)%
1117  3.32191% 140722276908360% (Xorg)%
826   3.20461% 140722276908360% (snapd)%
6727  2.31486% 140722276908360% (chrome)%
7596  0.934645% 140722276908360% (nautilus)%
1099  0.887751% 140722276908360% (mysqld)%
6524  0.723698% 140722276908360% (chrome)%
6709  0.660515% 140722276908360% (chrome)%
6567  0.526326% 140722276908360% (chrome)%
6496  0.506018% 140722276908360% (gnome-terminal-)%
Enter PID to kill: |

```

```

student@D001-38: ~/Desktop/Z141011200_LSP/Task-5

Press 'q' to quitPID    CPU%    Memory (kB)    Name
1452  4.0723% 140722276908360% (gnome-shell)%
1117  3.32191% 140722276908360% (Xorg)%
826   3.20461% 140722276908360% (snapd)%
6727  2.31486% 140722276908360% (chrome)%
7596  0.934645% 140722276908360% (nautilus)%
1099  0.887751% 140722276908360% (mysqld)%
6524  0.723698% 140722276908360% (chrome)%
6709  0.660515% 140722276908360% (chrome)%
6567  0.526326% 140722276908360% (chrome)%
6496  0.506018% 140722276908360% (gnome-terminal-)%
Enter PID to kill: 6524
Process 6524 terminated successfully

Press 'q' to quit or Enter to refresh: |

```